SOFTWARE ENGINEERING

MINI PROJECT

TEAM: H6

SRN: PES1UG24CS484, PES1UG24CS446,PES1UG25CS805,PES1UG24CS439

Hardware-Based Voice Assistant for Multimedia Controls

Technologies

- Raspberry Pi / ESP
- Python
- MediaPipe
- Microphone
- Multimedia player

Supported Commands

- Play
- Pause
- Resume
- Volume Up
- Volume Down
- Next
- Previous

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

---

1. Introduction

1.1 Purpose

The purpose of this project is to develop a hardware-based voice assistant that allows users to control multimedia playback using simple voice commands. The system uses a microphone to receive voice input and processes the command using software running on a Raspberry Pi or ESP-based hardware platform.

The system provides basic multimedia controls such as playing, pausing, resuming, increasing and decreasing volume, moving to the next media item and returning to the previous media item.

The project aims to provide a convenient hands-free method of controlling multimedia without requiring the user to physically interact with a keyboard, mouse or touchscreen.

1.2 Scope

The system is designed to provide voice-based control of multimedia applications.

The scope of the system includes:

- Capturing the user's voice through a microphone.
- Recognizing predefined voice commands.
- Processing the recognized command.
- Mapping the command to the corresponding multimedia operation.
- Controlling media playback.
- Controlling multimedia volume.
- Handling invalid or unrecognized commands.
- Providing appropriate feedback when a command cannot be executed.
- Supporting hardware-based operation using Raspberry Pi or ESP-based hardware.

The system focuses on predefined multimedia commands rather than general-purpose conversational voice assistance.

1.3 Product Perspective

The proposed system acts as an interface between the user and a multimedia player.

The user provides a voice command. The microphone captures the command, the voice-processing module identifies the command, and the command-processing module determines the required action. The multimedia-control module then performs the corresponding operation.

The overall flow is:

User → Microphone → Voice Recognition → Command Processing → Multimedia Control → Media Player

---

1.4 Product Functions

The major functions of the system are:

1. Voice input acquisition.
2. Voice command recognition.
3. Command validation.
4. Play control.
5. Pause control.
6. Resume control.
7. Volume increase.
8. Volume decrease.
9. Next media control.
10. Previous media control.
11. Invalid command handling.
12. Error handling.
13. Hardware interaction.

1.5 Users

The primary user of the system is a person who wants to control multimedia playback using voice commands.

The system is intended to be simple enough that a user does not require technical knowledge to operate it.

1.6 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
UML	Unified Modeling Language
ESP	Embedded/IoT development platform
Media	Audio or video content
Voice Command	Spoken instruction given by the user

---

2. Overall Description

2.1 System Environment

The system consists of a hardware platform such as Raspberry Pi or ESP, a microphone for capturing voice commands, software modules for processing the commands, and a multimedia player for performing the requested actions.

Python is used for software implementation and command processing. MediaPipe can be incorporated as an additional interaction/processing component where required by the project design.

2.2 System Workflow

The system operates in the following sequence:

1. The user speaks a predefined command.
2. The microphone captures the voice.
3. The voice-processing module processes the input.
4. The system identifies the intended command.
5. The command is passed to the command-processing module.
6. The command-processing module validates the command.
7. The corresponding multimedia operation is selected.
8. The multimedia controller sends the required action to the media player.
9. The media player performs the requested operation.
10. If the command cannot be understood or executed, the system handles the error appropriately.

---

3. Functional Requirements

FR-01 — Voice Input

Requirement:
The system shall capture voice input from the user through a connected microphone.

Explanation:
The microphone acts as the input device through which the user provides commands.

FR-02 — Voice Command Recognition

Requirement:
The system shall recognize predefined multimedia voice commands provided by the user.

Explanation:
The system should identify commands such as Play, Pause, Resume, Volume Up, Volume Down, Next and Previous.

FR-03 — Play Media

Requirement:
The system shall start multimedia playback when the user provides the Play command.

Explanation:
When the system recognizes the Play command, it shall instruct the multimedia player to start playback.

FR-04 — Pause Media

Requirement:
The system shall pause the currently playing media when the user provides the Pause command.

Explanation:
The system should temporarily stop media playback without closing the media player.

FR-05 — Resume Media

Requirement:
The system shall resume media playback when the user provides the Resume command.

Explanation:
The system should continue playback from the point at which the media was paused.

FR-06 — Increase Volume

Requirement:
The system shall increase the multimedia volume when the user provides the Volume Up command.

Explanation:
The command processor identifies the command and sends an increase-volume request to the multimedia controller.

FR-07 — Decrease Volume

Requirement:
The system shall decrease the multimedia volume when the user provides the Volume Down command.

Explanation:
The system reduces the current multimedia volume when the corresponding command is recognized.

FR-08 — Next Media

Requirement:
The system shall move to the next available media item when the user provides the Next command.

Explanation:
The multimedia controller shall instruct the media player to move from the current media item to the next item.

FR-09 — Previous Media

Requirement:
The system shall move to the previous available media item when the user provides the Previous command.

Explanation:
The system shall request the media player to return to the previous media item.

FR-10 — Invalid Command Handling

Requirement:
The system shall reject commands that do not match the predefined multimedia command set.

Explanation:
If the user says something that the system does not recognize as a valid command, the system should not perform any multimedia operation.

FR-11 — Error Handling

Requirement:
The system shall provide an appropriate error indication when voice input, hardware or multimedia control cannot be processed.

Explanation:
The system should handle problems such as microphone failure, unavailable media or unrecognized commands without causing unintended multimedia operations.

---

4. Non-Functional Requirements

NFR-01 — Performance

The system shall process a valid voice command within a defined response time after the command has been captured.

Explanation:
The system should respond quickly enough that the user does not experience unnecessary delay.

NFR-02 — Usability

The system shall provide simple and easily understandable voice commands for multimedia control.

Explanation:
Users should be able to operate the system without requiring extensive training.

NFR-03 — Reliability

The system shall perform the requested multimedia operation correctly when a valid command is recognized.

Explanation:
A valid command should result in the corresponding action and should not randomly trigger another operation.

NFR-04 — Security

The system shall validate voice commands before executing multimedia operations.

Explanation:
Only commands belonging to the predefined command set should be processed.

NFR-05 — Maintainability

The system shall use separate modules for voice recognition, command processing and multimedia control.

Explanation:
This allows individual components to be modified or extended without requiring major changes to the entire system.

NFR-06 — Portability

The software design should support deployment on the selected Raspberry Pi or compatible embedded hardware platform.

---

5. Security

5.1 Security Objectives

SO-01 — Prevent Unauthorized Commands

The system should prevent invalid or unauthorized inputs from causing multimedia operations.

SO-02 — Protect Voice Input

The system should minimize unnecessary storage or exposure of voice input and process voice commands only for the intended multimedia-control functionality.

5.2 Security Requirements

SR-01 — Command Validation

The system shall validate a recognized command against the predefined list of supported multimedia commands before execution.

SR-02 — Invalid Input Protection

The system shall not perform a multimedia operation when an input cannot be successfully validated as a supported command.

---

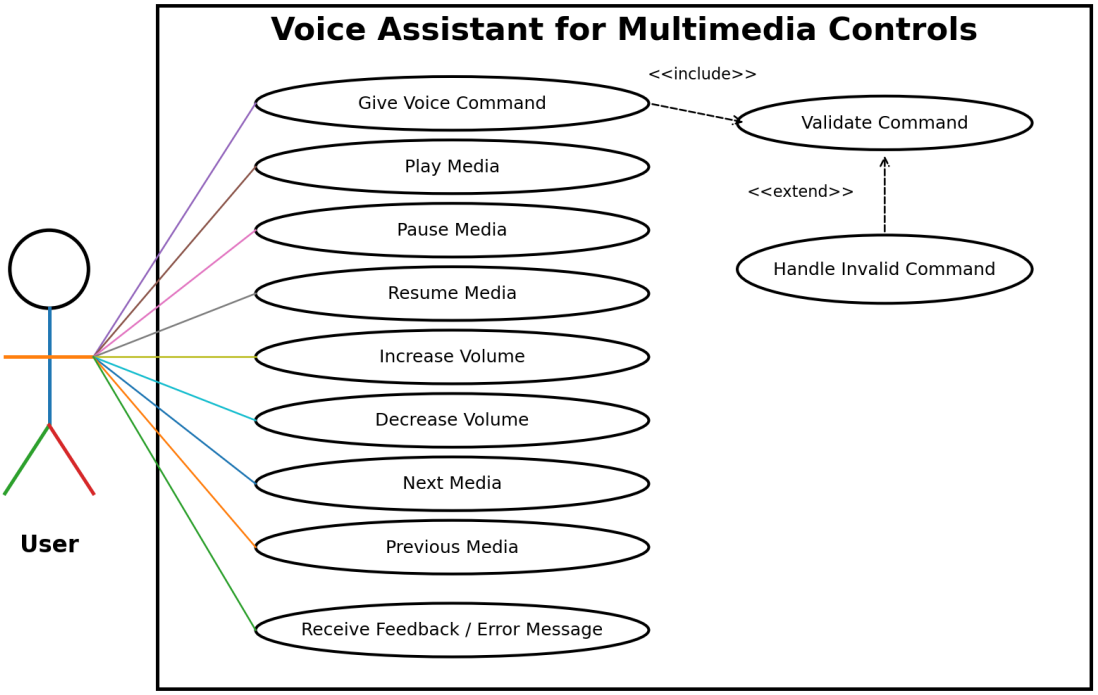
6. Use Case Description

Actor

User

Main Use Cases

- Give Voice Command
- Play Media
- Pause Media
- Resume Media
- Increase Volume
- Decrease Volume
- Next Media
- Previous Media
- Receive Error/Feedback



TEST PLAN

1. Introduction

1.1 Purpose

The purpose of the Test Plan is to define the testing approach used to verify that the hardware-based voice assistant satisfies its functional, non-functional and security requirements.

Testing will verify whether the system correctly recognizes predefined voice commands and performs the corresponding multimedia operations.

1.2 Scope

Testing will cover:

- Voice command recognition.
- Play functionality.
- Pause functionality.

- Resume functionality.
- Volume control.
- Next media functionality.
- Previous media functionality.
- Invalid command handling.
- Error handling.
- Performance.
- Reliability.
- Security validation.

---

2. Test Objectives

The main testing objectives are:

1. Verify that supported voice commands are correctly recognized.
2. Verify that each recognized command performs the expected multimedia operation.
3. Verify that invalid commands do not trigger unintended actions.
4. Verify that system errors are handled appropriately.
5. Verify the performance and usability of the system.
6. Verify the security requirements.
7. Verify traceability between requirements and test cases.

---

3. Testing Approach

The system will be tested using predefined inputs and expected outputs.

For functional testing, valid voice commands will be provided and the resulting multimedia action will be observed.

For negative testing, invalid or unsupported commands will be provided to verify that the system does not execute unintended actions.

For non-functional testing, performance, usability and reliability will be evaluated.

For security testing, command validation and invalid-input handling will be checked.

---

4. Test Environment

The test environment consists of:

- Raspberry Pi / ESP-based hardware

- Microphone
- Python environment
- Required voice-processing software
- Multimedia player
- Audio/video media files
- Network connection where required by the selected implementation

---

5. Security Validation

The security validation will verify that:

1. Only predefined commands are executed.
2. Invalid commands are rejected.
3. Unrecognized input does not trigger multimedia operations.
4. Voice input is handled according to the system's privacy requirements.
5. The command-processing module validates commands before sending them to the multimedia controller.

---

6. Requirement Traceability

The test plan will maintain traceability between SRS requirements and test cases.

Requirement	Requirement Description	Test Case
FR-01	Capture voice input	TC-01
FR-02	Recognize voice commands	TC-01
FR-03	Play media	TC-02
FR-04	Pause media	TC-03
FR-05	Resume media	TC-04
FR-06	Increase volume	TC-05
FR-07	Decrease volume	TC-06
FR-08	Next media	TC-07
FR-09	Previous media	TC-08

SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

1. Architecture Overview

The system follows a modular architecture where different responsibilities are divided into separate components.

The major components are:

1. User
2. Microphone
3. Voice Recognition Module
4. Command Processing Module
5. Multimedia Control Module
6. Volume Control Module
7. Media Player
8. Hardware Platform

The modular structure allows each component to perform a specific responsibility and makes the system easier to understand, test and maintain.

---

2. Component Descriptions

2.1 User

The user provides voice commands to control multimedia playback.

2.2 Microphone

The microphone captures the user's spoken command and provides the voice input to the voice-processing component.

2.3 Voice Recognition Module

This module processes the captured voice input and identifies the spoken command.

Its output is a recognized command that can be passed to the command-processing module.

2.4 Command Processing Module

This module receives the recognized command and determines which multimedia operation should be performed.

For example:

"Volume Up" → Increase Volume

"Pause" → Pause Media

2.5 Multimedia Control Module

This module acts as the interface between the command processor and the media player.

It sends the required control operation to the media player.

2.6 Volume Control Module

This component handles volume-related commands.

It processes:

- Volume Up
- Volume Down

2.7 Media Player

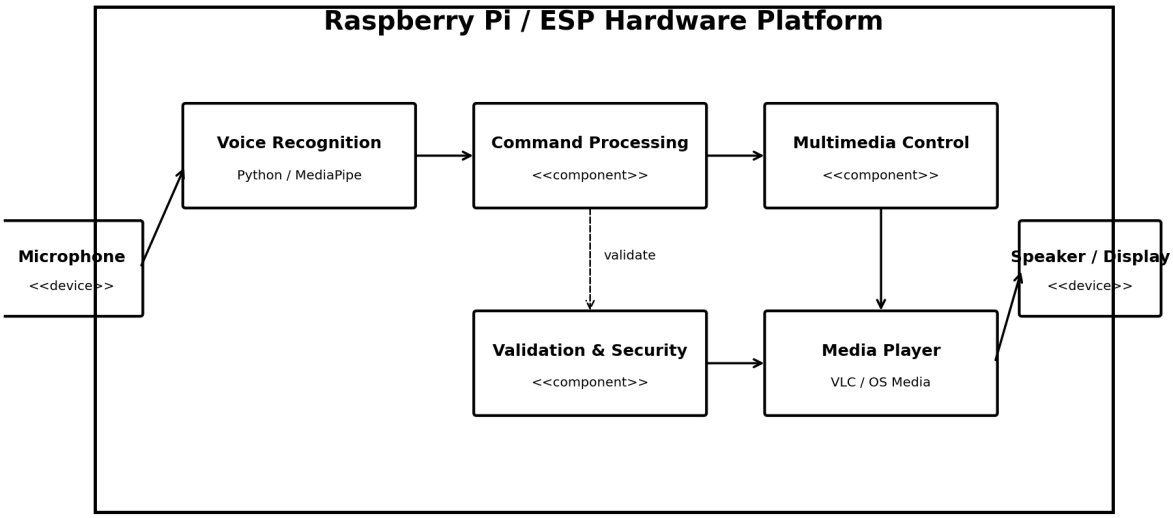
The media player performs the actual multimedia operation.

It is responsible for:

- Playing media
- Pausing media
- Resuming media
- Moving to next media
- Moving to previous media
- Applying volume changes

2.8 Hardware Platform

The Raspberry Pi or ESP-based platform provides the computing and hardware interface required to run the system and connect the input/output components.



---

3. Architectural Pattern

The system uses a **modular architecture**.

In this architecture, the system is divided into independent functional modules. Each module performs a specific task.

For example:

- Voice Recognition handles voice processing.
- Command Processing handles command interpretation.
- Multimedia Control handles media operations.
- Hardware Interface handles hardware interaction.

The modular approach makes the system easier to develop, test, maintain and extend.

If additional voice commands are required in the future, they can be added to the command-processing module without redesigning the entire system.

---

4. Architecture Requirement Traceability

Component	Related Requirement
Microphone	FR-01
Voice Recognition	FR-02

Component	Related Requirement
Command Processor	FR-02, FR-10
Multimedia Controller	FR-03 to FR-09
Error Handler	FR-11
Security/Validation Module	SR-01, SR-02
Hardware Platform	System-level requirements

---

5. Security Architecture

Security is incorporated into the command-processing flow.

The voice command is first recognized and then validated before execution.

The basic security flow is:

Voice Input → Recognition → Command Validation → Authorized Command → Multimedia Action

If the command is not recognized or does not match a supported command, it is rejected.

This prevents unsupported inputs from directly controlling the multimedia player.

The security architecture therefore focuses on:

- Input validation
- Command validation
- Invalid-command rejection
- Controlled multimedia access
- Appropriate handling of voice data

---

6. Design Overview

The design divides the system into functional modules.

Input Layer

Receives voice input from the microphone.

Processing Layer

Processes and recognizes the command.

Control Layer

Maps the command to the required multimedia operation.

Output Layer

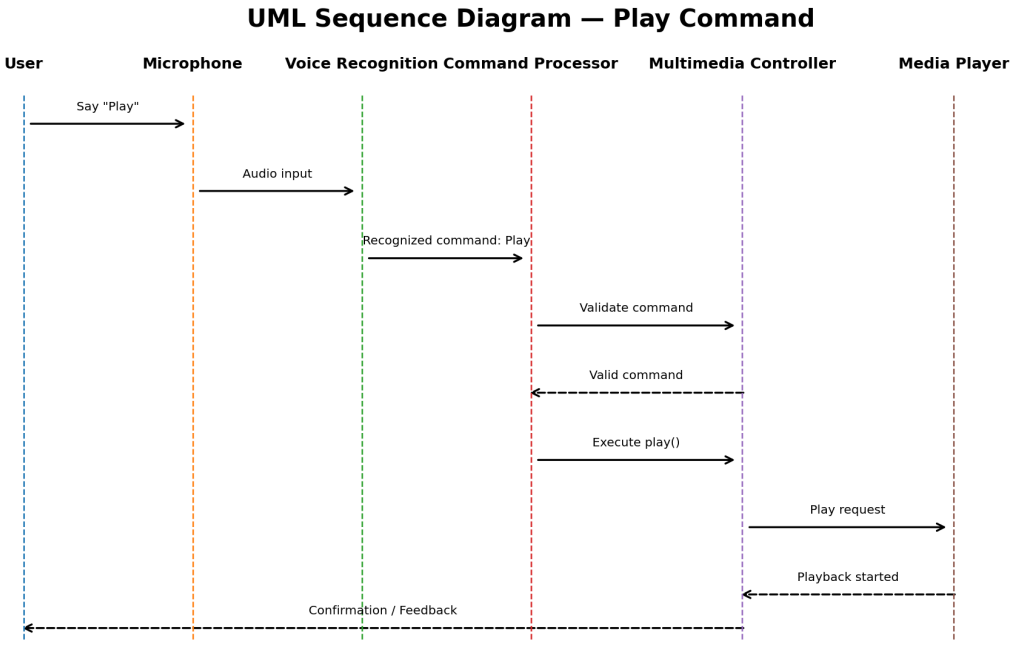
Executes the operation through the media player.

---

7. Sequence Diagram Explanation 1 — Play

Description

1. The user says the Play command.
2. The microphone captures the voice.
3. The voice recognition module processes the input.
4. The recognized command is sent to the command processor.
5. The command processor validates the command.
6. The command processor identifies the required Play operation.
7. The multimedia controller sends the Play request to the media player.
8. The media player starts playback.

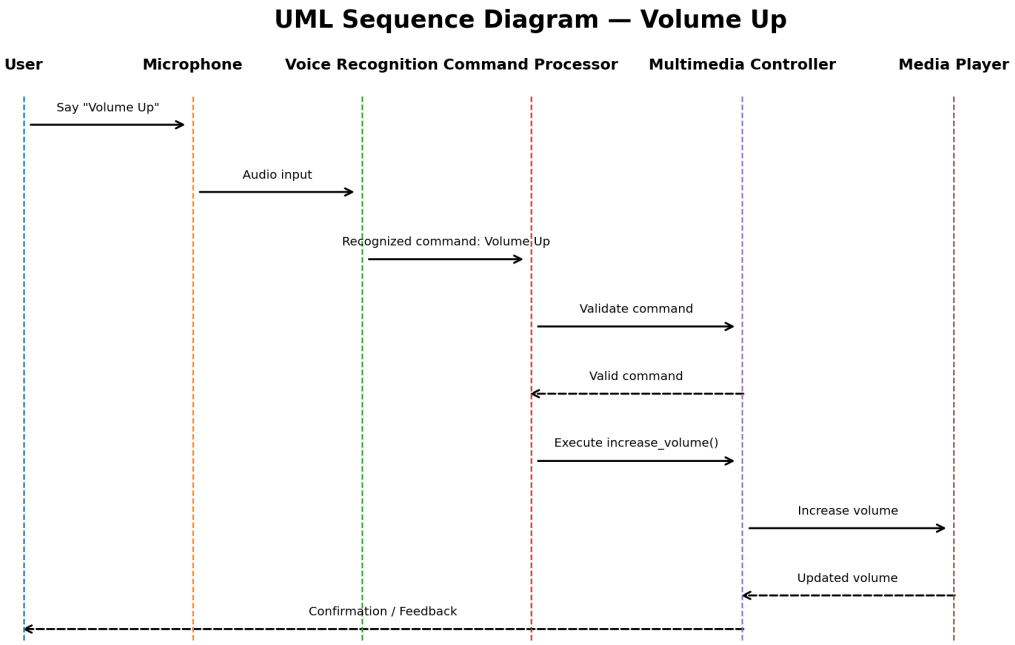


---

8. Sequence Diagram Explanation 2 — Volume Up

Description

1. The user says Volume Up.
2. The microphone captures the voice.
3. The voice recognition module identifies the command.
4. The command is passed to the command processor.
5. The command processor validates the command.
6. The command is mapped to the volume-control operation.
7. The multimedia controller requests an increase in volume.
8. The media player increases the volume.



---

9. API DESIGN

The system uses internal functions/interfaces to connect its modules.

9.1 `recognize_command()`

Purpose:
Recognizes the user's voice input.

Input:
Captured voice.

Output:
Recognized command.

9.2 `process_command(command)`

Purpose:
Determines the action associated with the recognized command.

Input:
Recognized command.

Output:
Corresponding multimedia operation.

9.3 `play_media()`

Purpose:
Starts media playback.

Input:
None.

Output:
Playback operation.

9.4 `pause_media()`

Purpose:
Pauses the currently playing media.

Input:
None.

Output:
Pause operation.

9.5 `resume_media()`

Purpose:
Resumes paused media.

Input:
None.

Output:
Resume operation.

9.6 `increase_volume()`

Purpose:
Increases the current multimedia volume.

Input:
None.

Output:
Updated volume.

9.7 `decrease_volume()`

Purpose:
Decreases the current multimedia volume.

Input:
None.

Output:
Updated volume.

9.8 `next_media()`

Purpose:
Moves to the next media item.

Input:
None.

Output:
Next media item.

9.9 `previous_media()`

Purpose:
Moves to the previous media item.

Input:
None.

Output:
Previous media item.

---

10. Error Handling

The system should handle errors without performing unintended multimedia actions.

Case 1 — No Voice Detected

If no voice input is detected, the system shall wait for another command.

Case 2 — Unrecognized Voice

If the voice cannot be recognized, the system shall reject the input and provide an appropriate indication.

Case 3 — Invalid Command

If the recognized input does not belong to the supported command set, the system shall not perform any multimedia action.

Case 4 — Microphone Failure

If the microphone is unavailable, the system shall indicate that voice input cannot be received.

Case 5 — Media Player Failure

If the multimedia player cannot perform the requested action, the system shall indicate that the operation could not be completed.

Case 6 — Hardware Failure

If the hardware interface is unavailable, the system shall provide an appropriate error indication rather than continuing with an unreliable operation.

TEST CASES

TC-01 — Voice Input Recognition

Requirement: FR-01, FR-02

Objective: Verify that the system can receive and recognize a supported voice command.

Input: "Play"

Steps:

1. Start the system.
2. Ensure the microphone is connected.
3. Speak "Play".
4. Observe the system.

Expected Result:
The system recognizes the command as Play.

TC-02 — Play Media

Requirement: FR-03

Input: "Play"

Expected Result:
The selected media starts playing.

TC-03 — Pause Media

Requirement: FR-04

Input: "Pause"

Expected Result:
Currently playing media is paused.

TC-04 — Resume Media

Requirement: FR-05

Input: "Resume"

Expected Result:
Paused media resumes playback.

TC-05 — Volume Up

Requirement: FR-06

Input: "Volume Up"

Expected Result:
The multimedia volume increases.

TC-06 — Volume Down

Requirement: FR-07

Input: "Volume Down"

Expected Result:
The multimedia volume decreases.

TC-07 — Next Media

Requirement: FR-08

Input: "Next"

Expected Result:
The next available media item starts/gets selected.

TC-08 — Previous Media

Requirement: FR-09

Input: "Previous"

Expected Result:
The previous media item is selected.
